DevOps Lab Experiment – 5

Introduction to Jenkins: What is Jenkins? Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

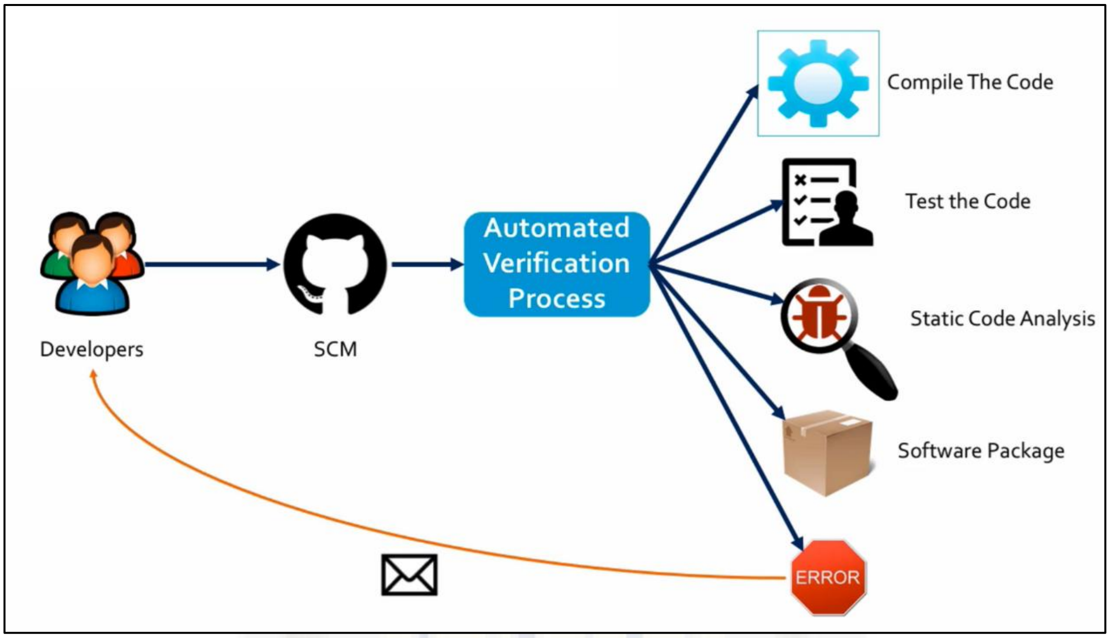
Introduction to Jenkins

Jenkins is a popular **open-source automation tool** used for Continuous Integration and Continuous Delivery. Similar tools include Bamboo, Circle CI, GitLab CI, Solano CI, and TeamCity etc.

Using Jenkins, we can automate any kind of task, not only continuous integration and delivery automation. Jenkins is an **open-source automation server** written in **Java**. It is a collection of plugins. All the features implemented in Jenkins is done via plugins. Using plugins, we can extend the features of Jenkins.

The **Continuous Integration (CI)** process works as follows:

1. **Developers** write code and push their changes to a **Source Code Management (SCM)** system such as **Git**.
2. Once the code is committed to **SCM**, an **Automated Verification Process** is triggered.
3. The automated process performs the following steps:
    - **Compile the code**
    - **Run automated test cases**
    - **Perform static code analysis**
    - **Generate a software package** (if all checks are successful)
4. If any errors are detected during compilation, testing, or analysis:
    - The process stops.
    - An **error notification** is sent back to the developers (usually via email or dashboard alerts).
5. If everything is successful:
    - A verified **software package** is created.
    - The build is marked as successful.



Contiguous Integration (CI) Workflow

Benefits of Continuous Integration

- Fast feedback to the developers
- Easy for the developers to find and fix issues
- Improves developers productivity
- Improves code quality, minimizes defects
- Its automates integration flow and makes frequent releases possible

Continuous Delivery and Continuous Deployment

After the Continuous Integration (CI) process, the tested software is deployed to **intermediate environments** such as:

- Development Environment
- Testing Environment
- QA (Quality Assurance) Environment
- Staging Environment (Pre-production environment that closely resembles the **Production Environment**)

After passing all verification stages, the software is finally deployed to the **Production Environment**.

The deployment is the practice of **frequently deploying** small code changes to production.

Continuous Delivery

- The software is always kept in a ***deployable state***.
- Deployment to production requires ***manual approval***.
- The release decision is a ***business decision***.
- The deployment process is automated, but production release is controlled.

Continuous Deployment

- Every successful build is **automatically deployed** to production.
- No manual approval is required.
- Deployment to production is fully automated.
- Code changes are released frequently and continuously.

CI = Build + Test + Verify + Package
CD = Deploy to environments such as
QA – Functional, Integration, and System Testing
Staging – Production-like environment, User Acceptance Testing (UAT), and final verification before release
Production – Live environment where real users access the application

Continuous Integration ensures code quality through automated building and testing.

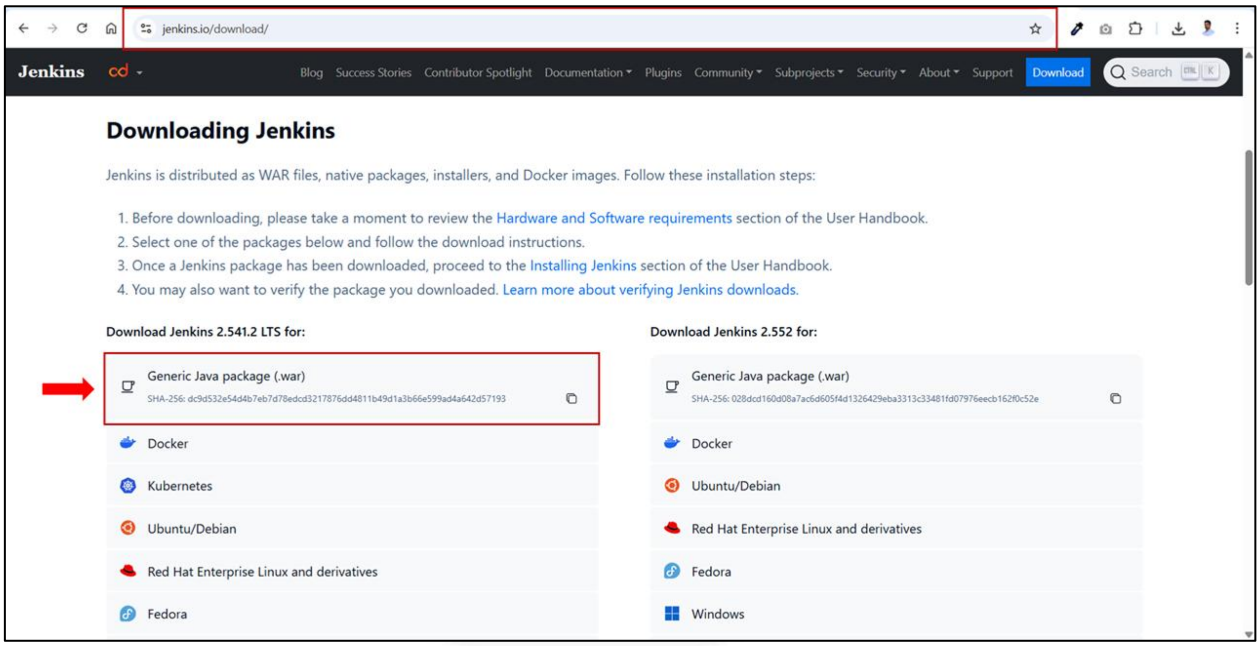
Continuous Delivery ensures the software is always ready for release.

Continuous Deployment automatically releases every successful build to production.

Jenkins Installation and Configuration

Step 1: Download Jenkins

1. Open a web browser and navigate to the official Jenkins download page https://www.jenkins.io/download/.
2. Create a folder named **Jenkins** in any drive on your local machine.



3. On the Jenkins download page, under **Downloading Jenkins**, select the latest **LTS** version.
4. Click on **Generic Java package (.war).** The file **jenkins.war** will start downloading.
5. Save the file in the **Jenkins** folder.

Step 2: Start (Run) the Jenkins server

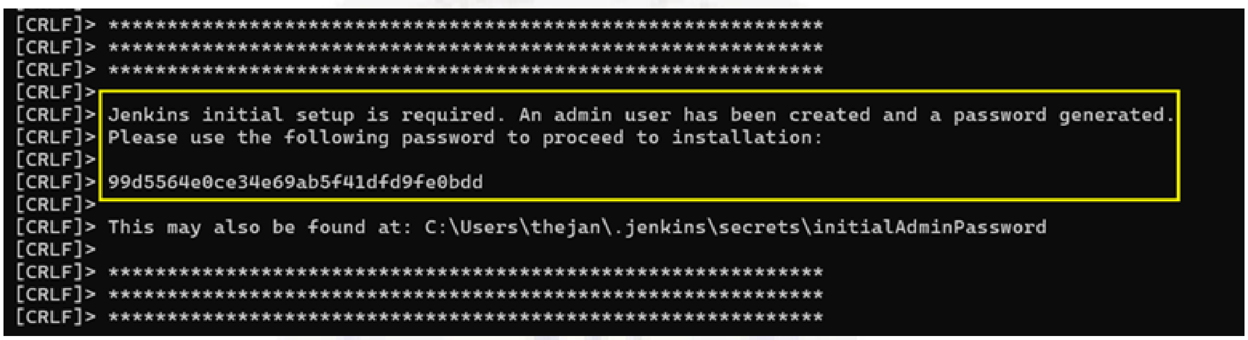
1. Navigate to the **Jenkins** folder, press **Alt + D**, type **cmd** in the address bar and press **Enter** to open Command Prompt (CMD) inside Jenkins folder.

2. Type the command **java -jar Jenkins.war** and execute it to run the Jenkins server.

3. The **initial admin password** is required for **security reasons** when you first install **Jenkins**. To copy the password, do not press **Ctrl + C**, as it will stop the server.

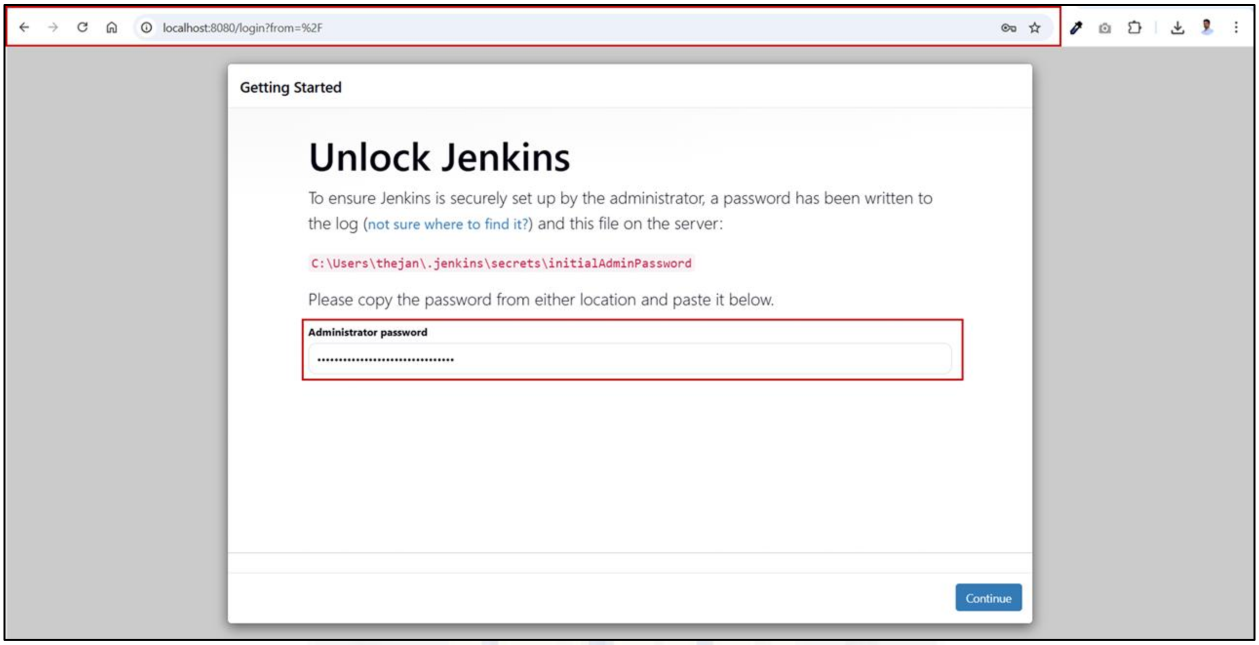
OR

Navigate to **C:\Users\<username>\.jenkins\secrets**, open the **initialAdminPassword** file in Notepad, and copy the password.

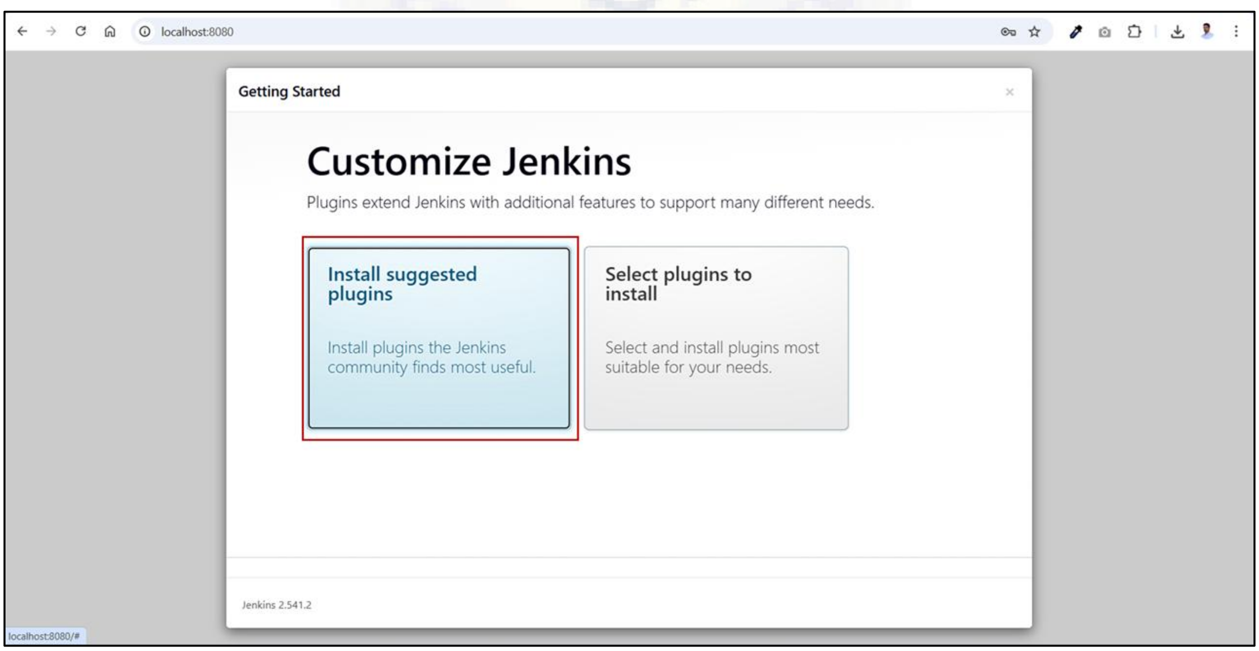


Step 3: Accessing the Jenkins Web Interface to Configure it

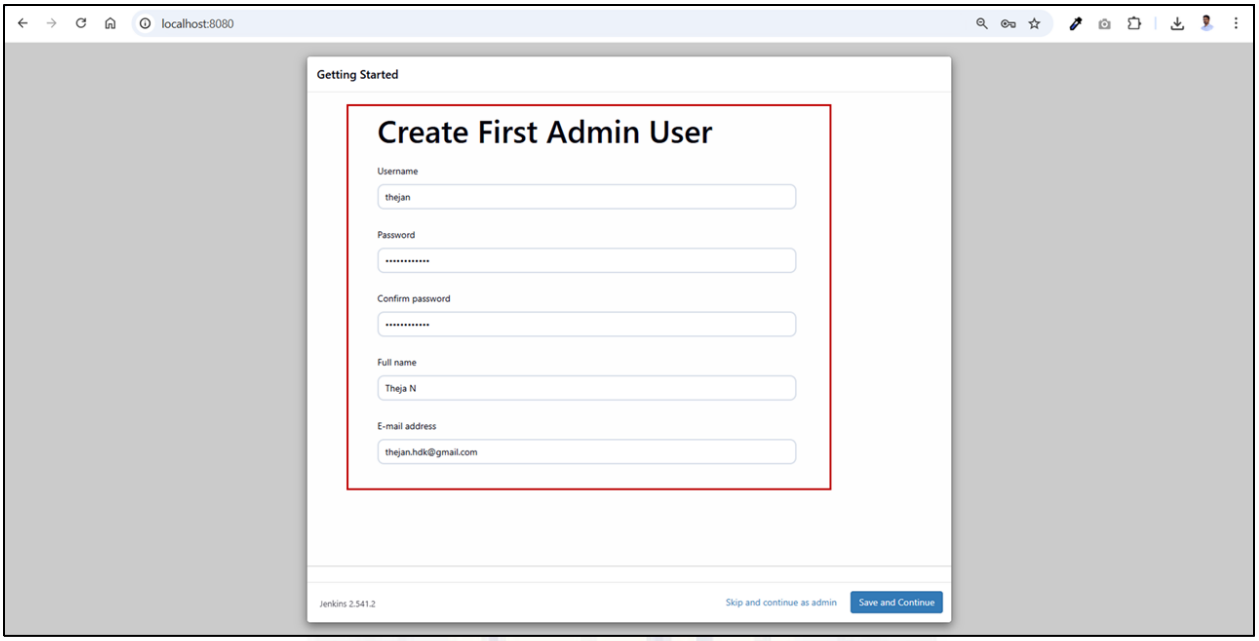
1. Open a web browser and go to http://localhost:8080 and the **Unlock Jenkins** page will appear.
2. Enter the password into the **Administrator password** field in the browser.
3. Click **Continue**.



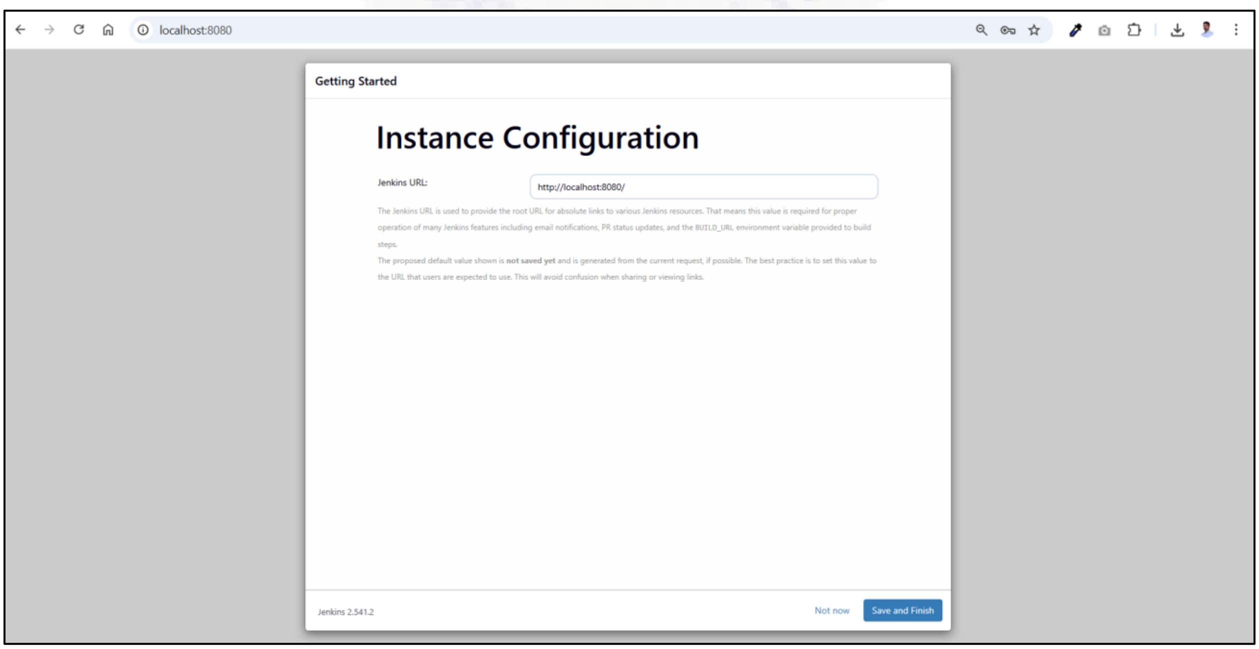
4. Select **Install suggested plugins.** Jenkins automatically installs a set of essential and commonly used plugins required for basic CI/CD functionality.

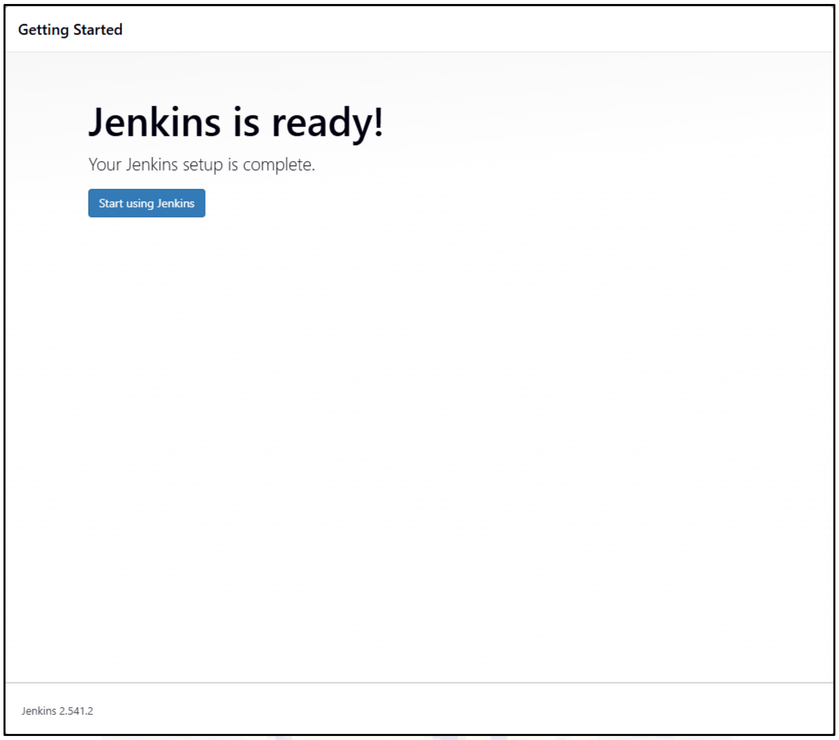


5. Enter the required details (**Username, Password, Full Name, and Email Address**), then click **Save and Continue.**



6. Click **Save and finish.**





When you see the message Jenkins is ready! Your Jenkins setup is complete., it means the installation and configuration are successful.

Viva Questions

1. What is Jenkins?
2. What is Continuous Integration?
3. What is Continuous Delivery?
4. What is the difference between Continuous Delivery and Continuous Deployment?
5. What is SCM? Give examples.
6. What is an artifact?
7. What is the default port number of Jenkins?
8. Where is the initialAdminPassword file located?
9. What are QA, Staging, and Production environments?
10. Can Jenkins be used for deployment? Explain.
11. What are the benefits of Continuous Integration?
12. Why should we deploy small changes frequently?